

DATACOPILOT

AI-Driven AutoML & Conversational Insights Engine

Submitted For: ABB Technical Challenge

Date: May 20, 2026

Evaluation Focus: Innovation, Technical implementation, Industrial Relevance, Scalability & Robustness

Scope: End-to-End AutoML Lifecycle

1. Executive Summary

DataCopilot is a production-ready Automated Machine Learning (AutoML) platform that automates the entire data science pipeline—from raw data ingestion to interactive, explainable model deployment. Unlike black-box AutoML tools, DataCopilot combines high-fidelity statistical evaluations with state-of-the-art SHAP (SHapley Additive exPlanations) values and an LLM-driven Chat Copilot, enabling domain experts to train, evaluate, visualize, and query complex machine learning models using natural language. The architecture is split into a robust FastAPI backend running parallel async task workers and a high-performance Next.js frontend with dark-mode analytics dashboards.

2. Inputs Considered

The DataCopilot system is engineered to handle multiple categories of inputs. These represent the starting parameters required to run the automated pipeline and initialize the intelligence modules.

2.1 Data and Schema Inputs

- **File Formats:** Structured tabular datasets in standard comma-separated (.csv) or JavaScript Object Notation (.json) files.
- **Schema Dimensions:** Arbitrary row count (\$N\$) and feature column count (\$D\$). Features can consist of mixed types (integers, floats, objects, booleans, timestamps).
- **Target Variable Inference:** The target column can be specified by the user or automatically inferred by mapping headers against semantic target hints (e.g., *survived*, *target*, *label*, *class*, *price*, *churn*).
- **Temporal Columns:** Date/time features, identified automatically by scanning columns for datetime data types or headers matching temporal keywords (e.g., *date*, *time*, *timestamp*, *year*).

2.2 System & Operational Inputs

- **Secure Key Store:** Firebase Admin Service Account credentials (serviceAccountKey.json) used to authenticate database operations with the Firestore DB.
- **Language Model APIs:** Client-side connection parameters and API keys for Google Gemini Pro (via google-genai) and Groq (Llama-3 models) to generate natural language explanations and handle user chat queries.
- **Workspace Directories:** A dedicated local workspace path (./workspace/uploads) on the backend server acts as a secure, sandboxed cache directory for active dataset sessions.

- **HTTP Header Configs:** Allowed CORS origins loaded from environment variables (.env) to restrict or permit connections from authorized frontend hosts.

3. Core AutoML Process to be Followed

DataCopilot executes an orchestrated 9-step pipeline to transform raw, noisy datasets into fully evaluated models backed by explainable artificial intelligence. The flowchart of operations spans data ingestion, statistical analysis, preprocessing, training, evaluation, and explanation.

Step 1: Upload	Step 2: Profile	Step 3: Task Detect
FastAPI endpoints validate and store raw uploads in a local secure sandbox. Establishes a Firestore session state.	Computes null ratios, data types, duplicate rows, numeric stats, and categorical frequencies.	Auto-detects target, temporal indices, class distributions, and evaluates class imbalance ratios.

Phase 1: Ingestion & Upload Management

When a client triggers file upload, the FastAPI server accepts the multipart file stream and restricts payloads based on file extensions. A unique UUID session ID is generated, representing the stateful record. The raw file is saved to the workspace, and a new session document is registered in Firestore. This step handles server-side storage and initiates the WebSocket connection for streaming pipeline progress.

Phase 2: Comprehensive Profiling & Diagnostics

Once written, the ingestion worker loads the file into memory using Pandas. It loops through all columns to generate a rich data profile dictionary. For every column, it calculates: (1) Missing value counts and percentage ratios, (2) native pandas data types, (3) summary statistics (mean, min, max, std) for numerical variables, and (4) value distributions (top 5 frequencies) for categorical variables. It also identifies duplicate records. This profile is serialized and returned to the client to drive visual dataset summaries.

Phase 3: Task Classification & Imbalance Engineering

A critical phase is automated machine learning task classification, which occurs without user intervention:

- Target Search:** The system scans column names against a predefined list of common target keywords (e.g. *target*, *label*, *class*, *y*, *churn*). If no matches are found, the last column is selected as the target.
- Temporal Scanning:** The system scans column names for date/time keywords. If a temporal column is detected, it registers the dataset as time-series oriented.
- Task Assignment:** If the target column is missing, the system assigns a **Clustering** task. If a temporal column is present, it assigns a **Time Series** task. For other cases, it checks the target data type: if it is numeric and contains more than 15 unique values, it assigns **Regression**; otherwise, it counts the classes—assigning **Binary Classification** for exactly 2 classes, and **Multi-class Classification** for more.
- Imbalance Diagnostic:** For classification tasks, it computes class frequency ratios. If the majority class represents over 75% of the samples, the `is_imbalanced` flag is set to true, triggering warnings in the user dashboard.

3. Core AutoML Process to be Followed (Continued)

Following task identification, the pipeline processes the data to make it compatible with machine learning algorithms and selects optimal candidate models for competitive cross-validation.

Step 4: Preprocessing	Step 5: Recommend	Step 6: Train (CV)
Drops high-null columns (>75%). Imputes numerals (median) and categoricals (mode). Encodes categories and standardizes numerals.	Initializes a list of candidate algorithms with parameters custom-tailored to the detected task.	Runs K-Fold, Stratified K-Fold, or Time Series Split. Streams training logs fold-by-fold via WebSockets.

Phase 4: Automated Feature Preprocessing & Transformation

Data preprocessing is applied sequentially to separate features (X) and target (y):

- **High-Sparsity Pruning:** Features with more than 75% missing values are dropped automatically.
- **Imputation:** Missing values in numeric columns are filled with their respective column's median. Missing values in categorical columns are filled with their column's mode. The exact details (value, count) are recorded.
- **Categorical Encoding:** High-cardinality categorical columns (>10 unique values) are Label Encoded. Low-cardinality categorical columns (<=10 unique values) are One-Hot Encoded into binary dummy columns, balancing feature dimensions.
- **Feature Standardization:** Numeric columns are standardized to zero mean and unit variance using Scikit-Learn's StandardScaler.
- **Target Variable Transformation:** For classification tasks, the target column is encoded via LabelEncoder. For regression, it is converted to numeric and missing entries are imputed with the median.

Phase 5: Candidate Model Recommendations

Rather than training a single model, DataCopilot recommends a list of models from Scikit-Learn and XGBoost, providing a clear technical justification for each. The model search space is customized based on the task type:

Task Type	Candidate Algorithms Included
Classification	XGBoost Classifier, Random Forest, Gradient Boosting, Logistic Regression, Naive Bayes, Neural Network (MLP), K-Nearest Neighbors (KNN)
Regression	XGBoost Regressor, Random Forest Regressor, Gradient Boosting Regressor, KNN, ElasticNet, MLP Neural Network
Clustering	K-Means (distance-based centroids), DBSCAN (density-based), Hierarchical Clustering (agglomerative linkages)
Time Series	Exponential Smoothing (Holt-Winters), ARIMA (auto-regressive), XGBoost Time Series (regression on temporal offsets)

Phase 6: Multi-Fold Cross-Validation Framework

To prevent overfitting, the models are trained using cross-validation. The number of splits is dynamically adjusted: $K = \min(5, \max(2, N / 30))$, ensuring that very small datasets do not fail during validation splits. For classification, the system uses StratifiedKFold to preserve target class proportions. For regression, standard KFold is used. For time-series, a temporal TimeSeriesSplit is used, preventing data leakage from future time steps. Progress and intermediate scores (Accuracy, R^2 , or Silhouette) are pushed to the WebSocket client fold-by-fold as they are computed.

3. Core AutoML Process to be Followed (Continued)

The final phases of the pipeline compute diagnostic evaluation metrics, explain the underlying model behavior using SHAP values, and initialize the conversational LLM interface.

Step 7: Evaluate	Step 8: SHAP (Explain)	Step 9: Visual & Chat
Evaluates all models on a holdout test set (80/20 split). Compares metrics and selects the best performer.	Calculates SHAP values to explain global feature importances. Falls back to tree/linear importances.	Generates heatmaps, histograms. Runs the LangChain LLM agent for interactive, natural language querying.

Phase 7: Rigid Model Evaluation & Holdout Testing

A separate 20% holdout test dataset is set aside to evaluate model generalization. The backend trains each candidate model on the training fold, predicts on the test set, and calculates a wide range of metrics: Classification models compute Accuracy, Precision, Recall, F1-Score, and ROC-AUC; Regression models calculate R², MAE, RMSE, and MAPE; Clustering models calculate Silhouette Scores and the Davies-Bouldin Index. For classification, the system also computes a complete Confusion Matrix and ROC curve coordinates for the best-performing model (based on CV score).

Phase 8: SHAP Interpretability & Feature Attribution

To provide model transparency, DataCopilot computes SHAP (SHapley Additive exPlanations) values for the best-performing model, identifying how much each feature contributes to predictions. Because SHAP calculations can be computationally intensive, a multi-tier fallback architecture is used:

- TreeExplainer:** Used for tree-based ensemble models (XGBoost, Random Forest, Gradient Boosting) to compute exact SHAP values quickly.
- KernelExplainer:** If the best model is non-tree-based (e.g., MLP, KNN, Logistic Regression), the system samples a small background dataset (up to 50 rows) to estimate SHAP values.
- Feature Importances:** If SHAP fails entirely, the system extracts the model's native `feature_importances_`.
- Coefficients:** For linear models, the system averages absolute model weights (`coef_`) to rank features.

The resulting top 15 features are sorted and stored to render feature attribution charts.

Phase 9: Conversational AI & Persistence Setup

When the pipeline finishes, the backend compiles the data profile, model scores, and SHAP features into a structured payload. This is passed to a background thread to generate an initial AI analysis using Google Gemini or Groq. The prompt instructs the LLM to explain the model choice, explain which features drive performance, point out any data anomalies, and offer business recommendations. This response is saved in Firestore as the session's `ai_insight` and sent to the client. Users can then ask follow-up questions in the chat interface, which is powered by LangChain and retrieves past messages from Firestore to maintain conversation context.

4. Expected Outputs

The DataCopilot system generates two primary types of outputs: a structured database state containing detailed machine learning metadata, and an interactive frontend dashboard UI.

4.1 Structured Firestore Session Schema

The ultimate output of the pipeline is a rich JSON document persisted in Firestore under a unique session_id. This structured record enables full reproducibility and is defined as follows:

```

{
  "uid": "firebase_user_uid", "session_id": "823e6659-693b-491a-a05e-d5d9b11fad00",
  "filename": "customer_churn.csv", "status": "done", "created_at": "2026-05-20T02:12:14Z",
  "ai_insight": "The best model is XGBoost (94.2% Acc)... Key drivers: MonthlyCharges, Tenure...",
  "steps": {
    "upload": { "status": "done", "data": { "rows": 7043, "cols": 21 } },
    "analyze": { "status": "done", "data": { "per_column": [...], "duplicate_rows": 0 } },
    "task": { "status": "done", "data": { "target_col": "Churn", "task_type": "Binary Classification" } },
    "preprocess": { "status": "done", "data": { "transforms": [...], "feature_names": [...] } },
    "recommend": { "status": "done", "data": { "models": [...] } },
    "train": { "status": "done", "data": { "cv_results": { "xgboost": { "mean": 0.942, ... } } } },
    "evaluate": { "status": "done", "data": { "metrics": [...], "confusion_matrix": [...] } },
    "shap": { "status": "done", "data": { "features": [ { "name": "Tenure", "importance": 0.354 }, ... ] } },
    "viz": { "status": "done", "data": { "correlation_matrix": [...], "histograms": {...} } }
  }
}
```

4.2 Next.js Dashboard Mockup

The Next.js frontend uses this state to render a premium dashboard, showing the pipeline progress, performance charts (ROC curves, heatmaps), SHAP feature importance, and an AI chat assistant. Below is the generated dashboard interface mockup:

